Criterion A - Problem Specification

Problem Statement

Small and medium business owners must regularly reconcile their internal ledgers with bank statements. The current process is manual and tedious taking 1-2 hours a month. It is also prone to human error. Transactions missing from one file but present in the other like mismatched dates, different amounts, missing entries, duplicated records etc. These mistakes are easy to overlook and lead to cashflow surprises, incorrect tax filings, and wasted time resolving disputes with banks or customers. Tally and similar platforms provide full-fledged accounting solutions with payroll, GST filing, and inventory. But they are paid,complex to learn and an overkill for small business owners who only need to reconcile their ledger and bank statements.

To solve this I will create a web application that will :

- Compare the two files and flag out these irregularities and generate a reconciliation report listing all unmatched entries
- The user will be able to upload the two files in PDF file format
- It will be free, easy to use with a minimal and distraction- free interface with sole focus on reconciliation

Computational Context

For the development of this application I will use :

- Python. Even though Java and C# are compiled into bytecodes, thus being more efficient at computative intensive tasks, the application is not expected to handle such a large volume of data to justify the added complexities and time for using them. And libraries like pandas (data manipulation and reconciliation logic), pdfplumber (extract tabular data from the PDF files) and rapidfuzz (row and column matching) are necessary for the application as they handle the PDF extraction and data manipulation directly in Python

- For the GUI React.js and Bootstrap will be used. This will allow the reconciliations to be done without reloading the page or calling the server again. And bootstrap has pre-built UI components like tables, modals, buttons etc therefore eliminating the need to write HTML and CSS from scratch

- I am making it as a web app for the following benefits - multi user access from different devices, no installation needed as users can log in from any browser and it will have easier maintenance and scalability for multi-user or multi-branch setups. And for this purpose I will be using flask as the web framework. I choose it over Django as it's lightweight and minimal and gives me full control over the application structure without the unnecessary complexity.

- PostgreSQL for the database as it handles multi-user access,strong data integrity through ACID transactions and has role-based access control so only authorised users can access the sensitive financial data

- SQLAlchemy as the ORM because it directly converts Python objects into SQL queries which is beneficial as: removes the need to write raw SQL, reduces risk of injection attacks and keeps the codebase consistent and maintainable.

Success criteria

- **File Upload** : The user will be able to upload the ledger and bank statement files in selectable (text-based) PDFs
- **Tolerance** : The user will be able to set absolute/percentage amount tolerance and date window. (Eg. ±3 days, difference up to 100 or percentage difference up to 3%)
- **Automatic Matching** : The app will match the transactions from the two uploaded files while applying the user-defined tolerances
- **Flagging system** : The app will flag the transactions present in one file but missing in other for the user to see
- **Reconciliation report** : The user will get a reconciliation report showing all numbers of matches, mismatches, total transactions. This report can also be exported in PDF format by the user
- **Performance** : The app will be able to process and reconcile up to 500 transactions in 10-20 seconds
- **Error Handling** : If an uploaded files is not a valid selectable PDF or is corrupted or has missing data the user will get an error message
- **User interface and feedback** : The user will get a loading/progression bar during the reconciliation to keep the user informed
- **Security and Privacy** : Each user will be required to log in from a unique email/username and set up a password for their account.
- **Compatibility** : The web app will be compatible on Windows macOS and Linux

(Word count - 467 Words)